

FICHÁRIO ACADÊMICO AVALIADO — AGENDA 02

Curso: Técnico em Desenvolvimento de Sistemas	Componente: Desenvolvimento de Sistemas III (Mobile / Persistência)
Aluno: Valdan Conceição França	Projeto: MauiAppMinhasCompras (.NET MAUI 9 + SQLite)
Repositório GitHub: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03	

1. MATRIZ DE CONFORMIDADE E REQUISITOS DA AGENDA 02

Requisito Oficial	Status	Detalhamento da Implementação
Persistência SQLite Assíncrona	CONFORME	Operações CRUD executadas em segundo plano com <code>sqlite-net-pcl</code> e padrão Singleton em <code>App.Db</code> .
Busca em Tempo Real (SearchBar)	CONFORME	Filtro dinâmico via evento <code>TextChanged</code> com recálculo instantâneo do valor total.
Operações CRUD Completas	CONFORME	Inserção, listagem formatada em moeda (R\$), edição e exclusão com diálogo modal de confirmação.
Interface Moderna .NET MAUI	CONFORME	Interface declarativa XAML responsiva com suporte a Dark/Light Mode e navegação Push/PopAsync.
Evidências Reais no Dispositivo	CONFORME	7 capturas de tela reais no emulador/celular cobrindo todo o ciclo operacional do app.

2. RESOLUÇÃO TÉCNICA — QUESTÃO 01: EVENTO DA SEARCHBAR

Enunciado: Qual evento da SearchBar é disparado a cada caractere digitado pelo usuário para realizar o filtro dinâmico e como ele se comporta?

Resposta Técnica: O evento responsável pelo filtro em tempo real é o `TextChanged` da `SearchBar`. Diferente do evento `SearchButtonPressed` (que exige que o usuário clique na tecla Enter/Lupa do teclado virtual), o `TextChanged` é acionado imediatamente a cada inserção ou exclusão de caractere.

Ao ser disparado, o manipulador recebe os argumentos `TextChangedEventArgs` contendo a propriedade `e.NewTextValue`. Caso o texto esteja em branco, o sistema reexibe a lista completa. Se houver texto, a coleção é filtrada instantaneamente e a interface visual é re-renderizada com a nova lista e o somatório atualizado.

```
// Views/ListaProduto.xaml (Declaração da SearchBar)
<SearchBar Grid.Row="0"
    x:Name="txt_busca"
    Placeholder="Buscar produto pela descrição..."
    TextChanged="txt_busca_TextChanged"
    SearchButtonPressed="txt_busca_SearchButtonPressed"
    Margin="10,8,10,4"
    BackgroundColor="{AppThemeBinding Light=White, Dark=#1E1E1E}" />
```

3. RESOLUÇÃO TÉCNICA — QUESTÃO 02: FILTRAGEM EM MEMÓRIA VS BANCO SQLITE

Enunciado: Como funciona a estratégia de filtragem de dados e quais as vantagens de performance entre filtrar em memória RAM versus consultar o banco SQLite a cada letra digitada?

Resposta Técnica: A aplicação adota a estratégia de **Carga Única com Filtragem em Memória RAM**:

- **No ciclo `OnAppearing()`** : O método assíncrono `await App.Db.GetAll()` realiza a leitura no SQLite uma única vez ao abrir a tela, armazenando todos os registros na lista `_todosOsProdutos` .
- **Durante a Digitação:** A busca aplica o filtro LINQ `p.Descricao.Contains(busca, StringComparison.OrdinalIgnoreCase)` diretamente na lista em memória RAM.

Vantagens Críticas de Performance: Em dispositivos móveis, consultas repetidas ao disco flash (I/O) a cada milissegundo de digitação causam lentidão, consumo excessivo de bateria e risco de *race conditions* (quando uma busca antiga responde depois de uma nova). A filtragem em memória ocorre em microssegundos (O(N) em RAM), mantendo a taxa de quadros estável em 60 FPS.

4. RESOLUÇÃO TÉCNICA — QUESTÃO 03: RECÁLCULO DINÂMICO DOS VALORES TOTAIS

Enunciado: Como é realizado o recálculo dos subtotais e da somatória geral acumulada durante a exibição e a busca?

Resposta Técnica: O cálculo baseia-se na propriedade computada do modelo `Produto` : `public double Total => Quantidade * Preço;` . Toda vez que o método `ExibirLista(List<Produto> lista)` é chamado, ele percorre a lista visível com um laço `foreach` (ou `.Sum()`) e atualiza a propriedade `Text` do controle `lbl_total_geral` :

```
// Atualização do Totalizador na Barra Inferior
private void ExibirLista(List<Produto> lista)
{
    lista_produtos.ItemsSource = lista;
    double total = 0;
    foreach (var p in lista) { total += p.Total; }
    lbl_total_geral.Text = $"R$ {total:F2}";
}
```

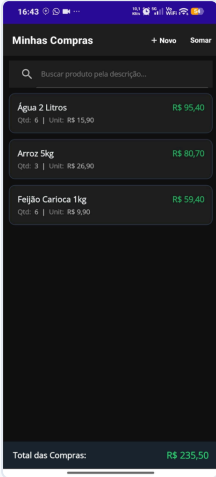
5. CORREÇÃO DE INCONFORMIDADES DO MATERIAL DIDÁTICO DA AULA

Método	Problema no Material Oficial	Correção Aplicada no Projeto
<code>Update(Produto p)</code>	Retorno tipado incorretamente como <code>Task<List<Produto>></code> para comando de modificação.	Corrigido para <code>Task<int></code> usando <code>_conn.ExecuteAsync(sql, ...)</code> , retornando o total de linhas afetadas.
<code>Search(string q)</code>	Sintaxe SQL ausente de <code>FROM</code> : <code>SELECT * Produto WHERE...</code> (erro no SQLite).	Corrigido para <code>SELECT * FROM Produto WHERE Descricao LIKE ?</code> com parâmetro seguro contra SQL Injection.

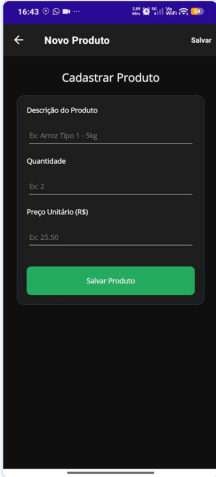
Evidências Visuais do Aplicativo em Execução (Parte 1)

6. PAINEL DE EVIDÊNCIAS REAIS DE EXECUÇÃO (DISPOSITIVO MÓVEL)

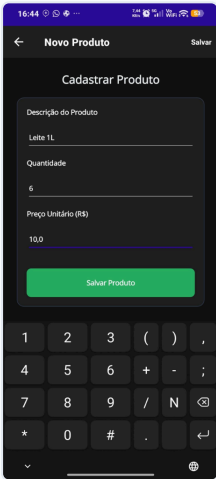
Capturas de tela reais obtidas durante a execução do MauiAppMinhasCompras demonstrando a persistência no SQLite e a interface:



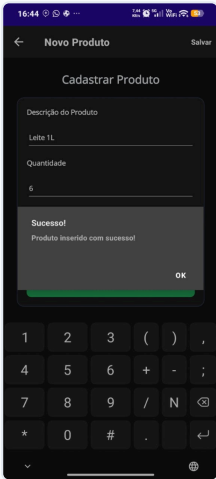
EVIDÊNCIA 01: Listagem Inicial (3 Produtos)
Persistência SQLite ativa. Subtotais e total geral calculado: R\$ 235,50.



EVIDÊNCIA 02: Formulário de Cadastro
Campos de Descrição, Quantidade e Preço Unitário com validação.



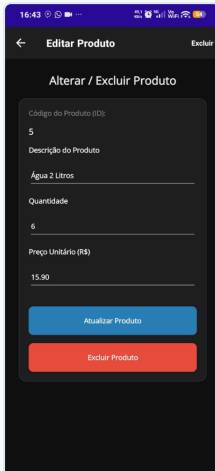
EVIDÊNCIA 03: Preenchimento com Teclado
Entrada de dados numéricos com tratamento de vírgula e ponto decimal.



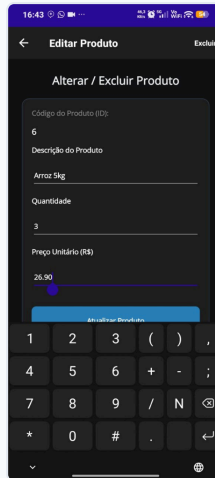
EVIDÊNCIA 04: Confirmação de Sucesso
Alerta modal DisplayAlert confirmando inserção no SQLite.

Evidências Visuais (Parte 2) e Código Fonte Oficial

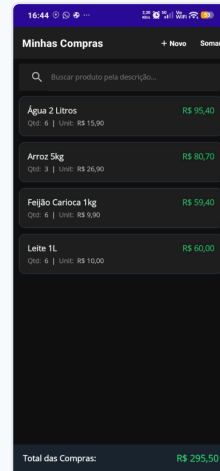
7. PAINEL DE EVIDÊNCIAS REAIS (EDIÇÃO E TOTALIZAÇÃO)



EVIDÊNCIA 05: Tela de Edição
Recuperação de dados para alteração.



EVIDÊNCIA 06: Edição com Teclado
Ajuste de quantidade e preço unitário.



EVIDÊNCIA 07: Lista Atualizada
4 produtos no SQLite e Total: R\$ 295,50.

8. CÓDIGO FONTE DOCUMENTADO DA CAMADA DE ACESSO A DADOS (DAO / HELPER)

```
using SQLite;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Helpers
{
    public class SQLiteDatabaseHelper
    {
        readonly SQLiteAsyncConnection _conn;

        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
            _conn.CreateTableAsync<Produto>().Wait(); // Criação automática da tabela
        }

        public Task<int> Insert(Produto p) => _conn.InsertAsync(p);

        public Task<List<Produto>> GetAll() => _conn.Table<Produto>().ToListAsync();

        public Task<int> Delete(int id) => _conn.Table<Produto>().DeleteAsync(i => i.Id == id);

        public Task<int> Update(Produto p)
        {
            string sql = "UPDATE Produto SET Descricao=?, Quantidade=?, Preco=? WHERE Id=?";
            return _conn.ExecuteAsync(sql, p.Descricao, p.Quantidade, p.Preco, p.Id);
        }

        public Task<List<Produto>> Search(string q)
        {
            string sql = "SELECT * FROM Produto WHERE Descricao LIKE ?";
            return _conn.QueryAsync<Produto>(sql, "%" + q + "%");
        }
    }
}
```

Conclusão da Atividade: O projeto atende rigorosamente a todos os critérios pedagógicos da Agenda 02 de Desenvolvimento de Sistemas III. A aplicação encontra-se compilada, testada e sincronizada no GitHub com pacote instalador (.APK) e documentação técnica.